Assignment 1: Quick Demonstration of Keras and TensorFlow Packages

1. Import required libraries

import tensorflow as tf

import keras

import numpy as np

2. Display TensorFlow and Keras versions

print("TensorFlow version:", tf.__version__)

print("Keras version:", keras.__version__)

Display available devices

print("Devices:", [d.device_type for d in tf.config.list_physical_devices()])

3. TensorFlow: Tensors and Automatic Differentiation

Create two tensors

a = tf.constant([[1., 2.], [3., 4.]])

b = tf.constant([[5., 6.], [7., 8.]])

Tensor addition

print("\nTensor addition:\n", (a + b).numpy())

Matrix multiplication

print("Matrix multiplication:\n", tf.matmul(a, b).numpy())

```
# Automatic differentiation using GradientTape
x = tf.Variable(3.0)

with tf.GradientTape() as tape:
    y = x ** 2 + 2 * x

# Calculate derivative
gradient = tape.gradient(y, x)

print("dy/dx of x^2 + 2x at x=3:", gradient.numpy())


# 4. Keras: Define, Compile, Train, Evaluate and Predict
# Generate sample dataset
np.random.seed(0)

X = np.random.rand(500, 4).astype("float32")
Y = (X.sum(axis=1) > 2).astype("float32")

# Define Sequential Neural Network Model
model = keras.Sequential([
    keras.layers.Input((4,)),
    keras.layers.Dense(8, activation="relu"),
    keras.layers.Dense(1, activation="sigmoid")
])

# Compile the model
```

```
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

# Display model architecture
model.summary()


# Train the model
model.fit(
    X,
    Y,
    epochs=15,
    batch_size=32,
    verbose=0
)


# Evaluate the model
evaluation = model.evaluate(X, Y, verbose=0)


print(
    "Evaluate [loss, accuracy]:",
    [round(float(v), 4) for v in evaluation]
)


# Make prediction
prediction = model.predict(
```

```
    np.array([[0.9, 0.8, 0.7, 0.6]]),
    verbose=0
)

print("Prediction for [0.9, 0.8, 0.7, 0.6]:", prediction.round(3))
```

```
# Assignment 2: Feedforward Neural Network (MNIST)
# Using Keras / TensorFlow with SGD Optimizer

# 1. Import required libraries
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.metrics import classification_report
from tensorflow.keras.datasets import mnist

# 2. Set random seeds
tf.random.set_seed(42)
np.random.seed(42)
```

```
# 3. Load MNIST dataset

(x_train, y_train), (x_test, y_test) = mnist.load_data()



# 4. Preprocess the dataset

# Flatten 28x28 images into 784 features

# Normalize pixel values between 0 and 1


x_train = x_train.reshape(-1, 784).astype("float32") / 255.0

x_test = x_test.reshape(-1, 784).astype("float32") / 255.0


print("Training data:", x_train.shape)

print("Testing data:", x_test.shape)



# 5. Build Feedforward Neural Network

model = keras.Sequential([

    layers.Input((784,)),

    layers.Dense(256, activation="sigmoid"),

    layers.Dense(128, activation="sigmoid"),

    layers.Dense(10, activation="softmax")

])



# 6. Display model architecture

model.summary()
```

```
# 7. Compile the model
model.compile(
    optimizer=keras.optimizers.SGD(learning_rate=0.01),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# 8. Train the model
H = model.fit(
    x_train,
    y_train,
    validation_data=(x_test, y_test),
    epochs=100,
    batch_size=128,
    verbose=2
)


# 9. Evaluate the model
loss, acc = model.evaluate(x_test, y_test, verbose=0)

print(f"\nTest loss: {loss:.4f}")
print(f"Test accuracy: {acc:.4f}")


# 10. Predict test data
pred = model.predict(x_test, verbose=0).argmax(axis=1)
```

```
print("\nClassification Report:")
print(classification_report(y_test, pred, digits=4))



# 11. Plot Training and Validation Graphs
plt.style.use("default")


fig, ax = plt.subplots(1, 2, figsize=(11, 4))



# Graph 1: Loss
ax[0].plot(H.history["loss"], label="train_loss")
ax[0].plot(H.history["val_loss"], label="val_loss")


ax[0].set_title("Training and Validation Loss")
ax[0].set_xlabel("Epoch")
ax[0].set_ylabel("Loss")
ax[0].legend()
ax[0].grid(alpha=0.3)



# Graph 2: Accuracy
ax[1].plot(H.history["accuracy"], label="train_acc")
ax[1].plot(H.history["val_accuracy"], label="val_acc")


ax[1].set_title("Training and Validation Accuracy")
ax[1].set_xlabel("Epoch")
```

```
ax[1].set_ylabel("Accuracy")
ax[1].legend()
ax[1].grid(alpha=0.3)


# Display and save graphs
plt.tight_layout()
plt.savefig("a2_curves.png", dpi=130)
plt.show()
```

```
# Experiment 3 - Image Classification using CNN (MNIST)


# 1. Import Libraries
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import mnist


# Set random seeds
tf.random.set_seed(42)
np.random.seed(42)
```

```
# 2. Load MNIST Dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# 3. Preprocess Image Data
x_train = x_train.astype("float32")[..., None] / 255.0
x_test = x_test.astype("float32")[..., None] / 255.0

y_train_c = keras.utils.to_categorical(y_train, 10)
y_test_c = keras.utils.to_categorical(y_test, 10)

print("x_train:", x_train.shape, "x_test:", x_test.shape)

# 4. Define CNN Model Architecture
model = keras.Sequential([
    layers.Input((28, 28, 1)),
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dropout(0.5),
    layers.Dense(10, activation="softmax"),
])

# Display Model Summary
model.summary()
```

```
# 5. Compile Model
model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

# 6. Train Model
H = model.fit(
    x_train,
    y_train_c,
    batch_size=128,
    epochs=10,
    validation_split=0.1,
    verbose=2
)

# 7. Evaluate Model Performance
loss, acc = model.evaluate(x_test, y_test_c, verbose=0)

print(f"\nTest loss: {loss:.4f}")
print(f"Test accuracy: {acc:.4f}")

# 8. Plot Training and Validation Loss / Accuracy

plt.style.use("default")

fig, ax = plt.subplots(1, 2, figsize=(11, 4))
```

```
# First Graph: Loss
ax[0].plot(H.history["loss"], label="train_loss")
ax[0].plot(H.history["val_loss"], label="val_loss")
ax[0].set_title("Model Loss")
ax[0].set_xlabel("Epoch")
ax[0].legend()
ax[0].grid(alpha=0.3)

# Second Graph: Accuracy
ax[1].plot(H.history["accuracy"], label="train_acc")
ax[1].plot(H.history["val_accuracy"], label="val_acc")
ax[1].set_title("Model Accuracy")
ax[1].set_xlabel("Epoch")
ax[1].legend()
ax[1].grid(alpha=0.3)

plt.tight_layout()
plt.savefig("a3_curves.png", dpi=130)
plt.show()

# 9. Predict First 10 Test Images
pred = model.predict(x_test[:10], verbose=0).argmax(axis=1)

# Display Sample Predictions
fig, ax = plt.subplots(1, 10, figsize=(12, 1.8))

for i in range(10):
```

```
    ax[i].imshow(x_test[i, :, :, 0], cmap="gray")
    ax[i].axis("off")
    ax[i].set_title(f"P:{pred[i]} T:{y_test[i]}", fontsize=8)

plt.tight_layout()
plt.savefig("a3_predictions.png", dpi=130)
plt.show()

print("Predicted:", pred.tolist())
print("Actual   :", y_test[:10].tolist())

# 10. Conclusion
print("\nCNN Image Classification implemented successfully!")
```

```
# Experiment 4: Autoencoder for Anomaly Detection (MNIST)
# Digit 1 = Normal, Other digits = Anomalies

# 1. Import Libraries
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    roc_auc_score
```

```
)
from tensorflow.keras.datasets import mnist


# Set random seeds
tf.random.set_seed(42)
np.random.seed(42)


# 2. Load MNIST Dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()


# 3. Preprocess Dataset
x_train = x_train.reshape(-1, 784).astype("float32") / 255.0
x_test = x_test.reshape(-1, 784).astype("float32") / 255.0


NORMAL = 1


# Train only on normal digit images
x_normal = x_train[y_train == NORMAL]


# Select test samples
rng = np.random.default_rng(42)


norm_idx = np.where(y_test == NORMAL)[0]


anom_idx = rng.choice(
    np.where(y_test != NORMAL)[0],
    size=150,
    replace=False
```

```
)

idx = np.concatenate([norm_idx, anom_idx])

x_eval = x_test[idx]
y_eval = (y_test[idx] != NORMAL).astype(int)

print("Normal training samples:", x_normal.shape[0])

print(
    "Test samples:", len(idx),
    "(normal:", (y_eval == 0).sum(),
    "anomalies:", (y_eval == 1).sum(), ")"
)

# 4. Build Autoencoder Model

# Encoder
inp = keras.Input((784,))

e = layers.Dense(128, activation="relu")(inp)
e = layers.Dense(64, activation="relu")(e)
code = layers.Dense(
    16,
    activation="relu",
    name="latent"
)(e)
```

```
# Decoder
d = layers.Dense(64, activation="relu")(code)
d = layers.Dense(128, activation="relu")(d)

out = layers.Dense(784, activation="sigmoid")(d)

autoencoder = keras.Model(
    inp,
    out,
    name="autoencoder"
)

# Display Model Summary
autoencoder.summary()

# 5. Compile Autoencoder
autoencoder.compile(
    optimizer="adam",
    loss="mse",
    metrics=["mae"]
)

# 6. Train Autoencoder
H = autoencoder.fit(
    x_normal,
    x_normal,
    epochs=30,
```

```
    batch_size=128,
    validation_split=0.1,
    shuffle=True,
    verbose=2
)

# 7. Calculate Reconstruction Error
def rec_error(x):
    reconstructed = autoencoder.predict(x, verbose=0)
    return np.mean((x - reconstructed) ** 2, axis=1)


# Calculate training reconstruction error
train_err = rec_error(x_normal)


# Set anomaly threshold
threshold = train_err.mean() + 2 * train_err.std()


# Calculate test reconstruction error
err = rec_error(x_eval)


# Predict anomalies
y_pred = (err > threshold).astype(int)


# 8. Display Evaluation Results
print(
    f"\nThreshold (mean + 2*std of training error): "
    f"{threshold:.5f}"
)
```

```
print(
    f"Mean reconstruction error - normal: "
    f"{err[y_eval == 0].mean():.5f} | "
    f"anomaly: {err[y_eval == 1].mean():.5f}"
)

print("ROC-AUC:", round(roc_auc_score(y_eval, err), 4))

print(
    "Confusion matrix [[TN FP],[FN TP]]:\n",
    confusion_matrix(y_eval, y_pred)
)

print("\nClassification Report:")

print(
    classification_report(
        y_eval,
        y_pred,
        target_names=["normal", "anomaly"],
        digits=4
    )
)

# 9. Plot Exactly Two Graphs

plt.style.use("default")
```

```
fig, ax = plt.subplots(1, 2, figsize=(11, 4))

# Graph 1: Autoencoder Training Loss
ax[0].plot(H.history["loss"], label="train_loss")
ax[0].plot(H.history["val_loss"], label="val_loss")

ax[0].set_title("Autoencoder training loss (MSE)")
ax[0].set_xlabel("Epoch")
ax[0].legend()
ax[0].grid(alpha=0.3)

# Graph 2: Reconstruction Error
ax[1].hist(
    err[y_eval == 0],
    bins=50,
    alpha=0.7,
    label="normal"
)

ax[1].hist(
    err[y_eval == 1],
    bins=50,
    alpha=0.7,
    label="anomaly"
)

ax[1].axvline(
    threshold,
```

```
    color="k",
    linestyle="--",
    label="threshold"
)

ax[1].set_title("Reconstruction error")
ax[1].set_xlabel("MSE")
ax[1].legend()
ax[1].grid(alpha=0.3)

plt.tight_layout()
plt.savefig("a4_error.png", dpi=130)
plt.show()

# 10. Display Original and Reconstructed Images

sel = (
    list(np.where(y_eval == 0)[0][:5]) +
    list(np.where(y_eval == 1)[0][:5])
)

rec = autoencoder.predict(x_eval[sel], verbose=0)

fig, ax = plt.subplots(2, 10, figsize=(12, 2.8))

for i, k in enumerate(sel):

    # Original images
```

```
    ax[0, i].imshow(
        x_eval[k].reshape(28, 28),
        cmap="gray"
    )
    ax[0, i].axis("off")

    # Reconstructed images
    ax[1, i].imshow(
        rec[i].reshape(28, 28),
        cmap="gray"
    )
    ax[1, i].axis("off")

    ax[0, i].set_title(
        "normal" if y_eval[k] == 0 else "anomaly",
        fontsize=8
    )

ax[0, 0].text(
    -0.3, 0.5, "input",
    transform=ax[0, 0].transAxes,
    rotation=90,
    va="center",
    ha="right",
    fontsize=8
)

ax[1, 0].text(
```

```
-0.3, 0.5, "output",
transform=ax[1, 0].transAxes,
rotation=90,
va="center",
ha="right",
fontsize=8
)


plt.tight_layout()
plt.savefig("a4_recon.png", dpi=130)
plt.show()


# 11. Conclusion
print("\nAutoencoder for Anomaly Detection implemented successfully!")
```

ASSIGNMNET NO 5

```
!pip install gensim


import gensim
from gensim.models import Word2Vec


print("Gensim Version:", gensim.__version__)
print("Gensim imported successfully!")


import nltk
```

```
nltk.download('gutenberg')

nltk.download('punkt')

nltk.download('punkt_tab')

nltk.download('stopwords')
```

```
from nltk.corpus import gutenberg


text = gutenberg.raw('carroll-alice.txt')


print("Dataset loaded successfully!")

print(text[:500])


# Assignment 5 - Continuous Bag of Words (CBOW) with Gensim Word2Vec

import re, nltk, warnings

warnings.filterwarnings("ignore")

import gensim

from gensim.models import Word2Vec

from nltk.corpus import stopwords

from nltk.tokenize import sent_tokenize, word_tokenize

# Step 1: dataset - "Alice's Adventures in Wonderland" (Project Gutenberg #11, same text
as

# 11-0.txt)

text = nltk.corpus.gutenberg.raw("carroll-alice.txt")

print("Characters in raw text:", len(text))

# Steps 2-6: pre-processing

sentences = sent_tokenize(text) # Step 5 (sentences)

stop_words = set(stopwords.words("english"))
```

```
data = []
for s in sentences:
 # Steps 2-3: remove special characters, digits, punctuation
 s = re.sub(r"[^A-Za-z\s]", " ", s)
 s = s.lower() # Step 4: lower case
 words = [w for w in word_tokenize(s) if w not in stop_words and len(w) > 1] # Steps 5-6
 if words: data.append(words)
print("Sentences after cleaning:", len(data))
print("Sample sentence:", data[10][:10])
# Step 7: train CBOW model (sg=0 -> CBOW, sg=1 -> Skip-gram)
model = Word2Vec(sentences=data, vector_size=100, window=5, min_count=2, sg=0,
workers=1,
 epochs=200, seed=1)
print("Gensim version:", gensim.__version__)
print("Vocabulary size:", len(model.wv))
print("Vector for 'alice' (first 8 dims):", model.wv["alice"][:8].round(4))
# Steps 8-9: similarity and prediction
print("\nSimilarity('alice','queen'):", round(float(model.wv.similarity("alice", "queen")), 4))
print("Similarity('alice','rabbit'):", round(float(model.wv.similarity("alice", "rabbit")), 4))
for w in ["alice", "queen", "king", "rabbit"]:
 print(f"\nTop-5 words most similar to '{w}':")
 for word, sc in model.wv.most_similar(w, topn=5): print(f" {word:<12}{sc:.4f}")
print("\nPredict output word for context ['mad','tea','party'] :")
for word, p in model.predict_output_word(["mad", "tea", "party"],
 topn=5): print(f" {word:<12}{p:.4f}")
print("Predict output word for context ['white','rabbit','hole']:")
for word, p in model.predict_output_word(["white", "rabbit", "hole"],
 topn=5): print(f" {word:<12}{p:.4f}")
```

```
# ============================================================

# ASSIGNMENT 6: OBJECT DETECTION USING TRANSFER LEARNING

# Model: YOLOv8n | Dataset: COCO128

# Includes freezing, fine-tuning, evaluation and visualization

# ============================================================


# Install required libraries

%pip install -q ultralytics matplotlib


# Import required libraries

import os

import numpy as np

import matplotlib.pyplot as plt

import torch

from pathlib import Path

from ultralytics import YOLO


# Reproducibility

torch.manual_seed(42)

np.random.seed(42)


device = 0 if torch.cuda.is_available() else "cpu"

print("Device:", device)
```

```
# ============================================================
# 1. LOAD PRETRAINED MODEL
# ============================================================

model = YOLO("yolov8n.pt")
print("Pretrained YOLOv8n model loaded.")

# ============================================================
# 2. STAGE 1: TRAIN WITH INITIAL LAYERS FROZEN
# ============================================================

print("\nStarting Stage 1: Frozen-layer training...")

model.train(
    data="coco128.yaml",
    epochs=5,
    imgsz=640,
    batch=8,
    freeze=10,
    device=device,
    workers=0,
    project="/content/assignment6_results",
    name="stage1_frozen",
    exist_ok=True,
    plots=True
)

# Obtain the actual directory used by Ultralytics
```

```
stage1_dir = Path(model.trainer.save_dir)
stage1_path = stage1_dir / "weights" / "best.pt"

# Fall back to last.pt if best.pt is unavailable
if not stage1_path.exists():
    stage1_path = stage1_dir / "weights" / "last.pt"

if not stage1_path.exists():
    raise FileNotFoundError(
        f"No trained weights found in: {stage1_dir / 'weights'}"
    )

print("Stage 1 completed.")
print("Stage 1 weights:", stage1_path)

# ============================================================
# 3. STAGE 2: FINE-TUNE ALL LAYERS
# ============================================================

print("\nStarting Stage 2: Fine-tuning...")

model = YOLO(str(stage1_path))

model.train(
    data="coco128.yaml",
    epochs=5,
    imgsz=640,
    batch=8,
```

```
    freeze=0,
    lr0=0.001,
    device=device,
    workers=0,
    project="/content/assignment6_results",
    name="stage2_finetune",
    exist_ok=True,
    plots=True
)

stage2_dir = Path(model.trainer.save_dir)
final_path = stage2_dir / "weights" / "best.pt"

if not final_path.exists():
    final_path = stage2_dir / "weights" / "last.pt"

if not final_path.exists():
    raise FileNotFoundError(
        f"No final weights found in: {stage2_dir / 'weights'}"
    )

print("Fine-tuning completed.")
print("Final model:", final_path)

# ============================================================
# 4. EVALUATE FINAL MODEL
# ============================================================
```

```
model = YOLO(str(final_path))

print("\nEvaluating final model...")

metrics = model.val(
    data="coco128.yaml",
    imgsz=640,
    batch=8,
    device=device,
    workers=0,
    project="/content/assignment6_results",
    name="evaluation",
    exist_ok=True,
    plots=True
)

print("\n========== EVALUATION RESULTS ==========")
print(f"Precision: {metrics.box.mp:.4f}")
print(f"Recall: {metrics.box.mr:.4f}")
print(f"mAP@50: {metrics.box.map50:.4f}")
print(f"mAP@50-95: {metrics.box.map:.4f}")

# ============================================================
# 5. OBJECT DETECTION AND BOUNDING-BOX VISUALIZATION
# ============================================================

print("\nRunning object detection on a sample image...")
```

```
image_url = "https://ultralytics.com/images/bus.jpg"


results = model.predict(
    source=image_url,
    conf=0.25,
    imgsz=640,
    device=device,
    save=False
)


# Draw bounding boxes, labels and confidence scores
annotated = results[0].plot()


# Convert BGR to RGB for Matplotlib
annotated = annotated[:, :, ::-1]


plt.figure(figsize=(12, 8))
plt.imshow(annotated)
plt.title("Object Detection Using Transfer Learning")
plt.axis("off")
plt.tight_layout()


output_image = "/content/assignment6_object_detection.png"


plt.savefig(output_image, dpi=150, bbox_inches="tight")
plt.show()


# ============================================================
```

```
# 6. DISPLAY DETECTED OBJECTS
# ============================================================

print("\n========== DETECTED OBJECTS ==========")

boxes = results[0].boxes

if boxes is not None and len(boxes) > 0:

    for box in boxes:

        class_id = int(box.cls.item())
        confidence = float(box.conf.item())
        coordinates = box.xyxy[0].tolist()

        print(
            f"Object: {model.names[class_id]} | "
            f"Confidence: {confidence:.2%} | "
            f"Bounding box: "
            f"{[round(v, 2) for v in coordinates]}"
        )

else:
    print("No objects detected.")

# ============================================================
# 7. FINAL OUTPUT
# ============================================================
```

```
print("\n========== ASSIGNMENT COMPLETED ==========")
print("Stage 1: Frozen-layer training")
print("Stage 2: Fine-tuning")
print("Final model:", final_path)
print("Detection image:", output_image)
print("Results directory:", stage2_dir)
print("Evaluation metrics and plots have been saved.")
```